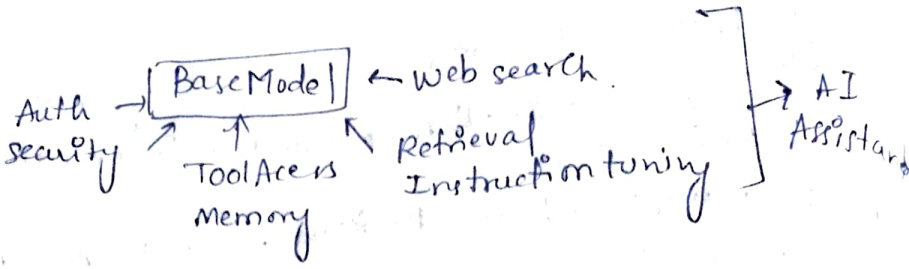
Base Model (vs chatgpt, claude, Gemini)

It is a model trained primarily to predict text



Inference (vs Training)

search → model
result

- After training
- The process of giving the result back to the user.
- Not heavy

Why models Hallucinate?

Fake fluency != real → confidence in language is not confidence in truth.

Hallucination occurs when an AI model generates information that appears plausible but is incorrect

possibility

- Insufficient info
- Ambiguous info
- Outdated knowledge
- False Assumptions
- Unreliable patterns (while training)
- Models are optimized to answer.
- Generation is probabilistic

Types of Hallucination

- Invented facts
- Invented citations
- Fake Reasoning etc.

Why models say idk?

- Assistant training
- System Instructions
- weak Patterns
- Tool Requirements
- prompt wording
- safety rules

How to avoid illusions
→ Ask to seperate facts from assumptions.
→ Ask for sources & use websearch etc

How tools extend the models?
↳ weather, location, mail, db, docs, files, websearch
Code execution etc

Web Search + LLMS
Retrieve + Generate.
(evidence) Useful response

Retrieval Agumented Generation → RAG. → Can be done. Private data

Note: → tools do not comply eliminate errors

Self Awareness:
→ Training
→ Context → Ability to connecting few dots
→ System prompt / Instructions
→ Tools

Secret Language of LLMS: → (Numbers) Tokens

→ computers cannot directly understand words
They understand numbers

Namaste AI is Amazing → each part of this
[78 12 34 89] → Token Id
Token

LLMS → [sequence of TokenId]

Token: A unit of text defined by Tokenizer.
↳ A machine that converts into tokens
↳ Algorithms that breaks strings to pieces of tokens

multiple characters

Why models use Subword tokenization

How tokenizer works:

Why model use subword tokenization:

The model tries to break the words into reusable token
if u write something simple He is a boy but untrustable
1 2 3 4 1 2 3

word tokenization creates a very lengthy vocabulary
ex untrust undetermined trustable trusty etc

Character tokenization create a very long sequence of array of tokens
[every char] → [token]

To maintain a balance we use
subword tokenization

Vocabulary and tokenId:

Every tokenizer has a vocabulary
↓
collection of tokens that the tokenizer uses to map

Token	Id
He	122
fn	32
ten	29

BPE → Byte Pair Encoding → Build a vocabulary by repeatedly merging frequently occurring neighbouring pieces

low
lower
lowest
→ (lo) → Token → 15
(low) → 18

New
Newset
New → 187

Bit → 01
L(ascii) → 01101010
Byte → 23

More like BPE → Word Piece Unigram

New models → have great vocabulary in tokenizer
So it will have very less number of tokens

Languages:

English
Hindi
Hinglish
Mixed

Tokenizer sees each and every string as different tokens

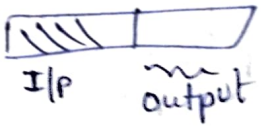
Special Tokens
→ System Instructions
→ start of msg
→ end of msg

After breaking into token —encode→ + special tokens → LLM
user ← res [_ _ _]
decode sequence of token Ids

Context window: →
It is the amount of tokenized information a model can process with in a request

Context can include:
System Instruction
Retrieved Information
User prompt (current)
previous messages
Tool output (calc/ weather etc)

Context window is shared btwn what you send and what the model generates



What happens if context window fills:

Rejecting the request
Truncating the Input (older chats)
Summarizing prev info
Retrieving only relavent sections → RAG.

Every model has its own Maximum Context Length

⇒

Longer prompt ↓= Better Results

Q) Explain closure in JS who understands functions but has never seen lexical env. keep it under 250 words

Tokens directly affect API cost